

Problem Set 3

Multi-Dimensional Packet Scrambler

Kamna Srivastava

August 2026

1 Objective

The objective of this practical is to work with Python lists, slicing, in-place modification, and multiple assignment unpacking.

The program performs four stages:

1. Input Validation and Test Data
2. Middle-Out Swap
3. In-Place Correction
4. Memory Integrity Check

2 Stage 1: Input Validation

The packet is defined as a list containing 10 integer values including two occurrences of zero.

```
packet = [5, 12, 0, 8, 21, 34, 7, 19, 0, 3]
```

```
if packet and len(packet) >= 10:
    print("Validation - passed - Processing - packet ...")
else:
    print("Validation - failed : - packet - is - empty - or - too - short.")
```

The validation checks whether the packet is not empty and contains at least 10 elements.

3 Stage 2: Middle-Out Swap

The packet is divided into two halves using slicing.

```
midpoint = len(packet) // 2

front_half = packet[:midpoint]
back_half = packet[midpoint:]
```

```
scrambled = back_half[::-1] + front_half
```

The front half is:

```
[5, 12, 0, 8, 21]
```

The back half is:

```
[34, 7, 19, 0, 3]
```

After reversing the back half and joining it with the front half:

```
[3, 0, 19, 7, 34, 5, 12, 0, 8, 21]
```

4 Stage 3: In-Place Correction

The middle index is calculated and SYNC-BIT is inserted when the value at the middle index is an integer.

```
middle_index = len(scrambled) // 2
```

```
if type(scrambled[middle_index]) is int:
    scrambled.insert(middle_index + 1, "SYNC-BIT")
```

After inserting SYNC-BIT:

```
[3, 0, 19, 7, 34, 5, "SYNC-BIT", 12, 0, 8, 21]
```

All zeros are then removed.

```
while 0 in scrambled:
    scrambled.remove(0)
```

The final scrambled list is:

```
[3, 19, 7, 34, 5, "SYNC-BIT", 12, 8, 21]
```

5 Stage 4: Memory Integrity Check

Multiple assignment unpacking is used to separate the header, body, and footer.

```
first, *middle, last = scrambled
```

```
print("Header:", first)
print("Footer:", last)
print("Body length:", len(middle))
```

The results are:

```
Header: 3
```

```
Footer: 21
```

```
Body length: 7
```

The original packet remains:

```
[5, 12, 0, 8, 21, 34, 7, 19, 0, 3]
```

6 Complete Python Program

```
packet = [5, 12, 0, 8, 21, 34, 7, 19, 0, 3]

print("——-Stage-1:-Input-Validation-——")

if packet and len(packet) >= 10:
    print("Validation-passed.-Processing-packet...")
else:
    print("Validation-failed:-packet-is-empty-or-too-short.")

print("Original-packet:", packet)

print("\n——-Stage-2:-Middle-Out-Swap-——")

midpoint = len(packet) // 2

front_half = packet[:midpoint]
back_half = packet[midpoint:]

scrambled = back_half[::-1] + front_half

print("Front-half:", front_half)
print("Back-half:", back_half)
print("Scrambled:", scrambled)

print("Is-packet-same-as-front_half?", id(packet) == id(front_half))

print("\n——-Stage-3:-In-Place-Correction-——")

middle_index = len(scrambled) // 2

if type(scrambled[middle_index]) is int:
    scrambled.insert(middle_index + 1, "SYNC-BIT")

print("After-SYNC-BIT-insertion:", scrambled)

while 0 in scrambled:
    scrambled.remove(0)

print("After-removing-zeros:", scrambled)

print("\n——-Stage-4:-Memory-Integrity-Check-——")

first, *middle, last = scrambled

print("Original-packet:", packet)
print("Final-scrambled-packet:", scrambled)
```

```
print("Header:", first)
print("Footer:", last)
print("Body length:", len(middle))
```

7 Output

```
--- Stage 1: Input Validation ---
Validation passed. Processing packet...
Original packet: [5, 12, 0, 8, 21, 34, 7, 19, 0, 3]

--- Stage 2: Middle-Out Swap ---
Front half: [5, 12, 0, 8, 21]
Back half: [34, 7, 19, 0, 3]
Scrambled: [3, 0, 19, 7, 34, 5, 12, 0, 8, 21]
Is packet same as front_half? False

--- Stage 3: In-Place Correction ---
After SYNC-BIT insertion:
[3, 0, 19, 7, 34, 5, 'SYNC-BIT', 12, 0, 8, 21]

After removing zeros:
[3, 19, 7, 34, 5, 'SYNC-BIT', 12, 8, 21]

--- Stage 4: Memory Integrity Check ---
Original packet: [5, 12, 0, 8, 21, 34, 7, 19, 0, 3]
Final scrambled packet:
[3, 19, 7, 34, 5, 'SYNC-BIT', 12, 8, 21]
Header: 3
Footer: 21
Body length: 7
```

8 Conclusion

The practical demonstrates Python list manipulation using slicing, insertion, removal, and multiple assignment unpacking. The original packet remains unchanged while the scrambled list is modified.